Vibe Coding Chat History

Tug of War Project

Course	Software Engineering Lab — PES University
Student Name	Nishanth T Chigilar
Repository	56_tug_of_war (Pygame Tug-of-War Game)
Document Scope	Complete AI prompt and response transcript provided for Lab 4

Transcript note: This document replaces the previous report content and records the conversation in the order supplied, including the Python implementations and the final confirmation that the game runs smoothly.

User / *Initial debugging and modularization request*

I am working on my Python Pygame Tug-of-War lab project (`56_tug_of_war`). The codebase is broken: keypress event handling for Player 1 ('A') and Player 2 ('L') does not pull the rope properly, the rope position moves off the screen, and win conditions are missing. Help me fix these bugs and modularize the code into `main.py`, `game/player.py`, `game/rope.py`, and `game/game_engine.py`.

Gemini / *Bug fixes and modular architecture*

Here is the bug fix and modular architecture for your Tug of War Pygame application:

game/player.py

```
import pygame

class Player:
    def __init__(self, name: str, key: int, color: tuple, side: str):
        self.name = name
        self.key = key
        self.color = color
        self.side = side

    def is_pulling(self, event: pygame.event.Event) -> bool:
        """Check if the user pressed their designated key."""
        return event.type == pygame.KEYDOWN and event.key == self.key
```

game/rope.py

```
import pygame

class Rope:
    def __init__(self, screen_width: int, win_threshold: int = 220):
        self.screen_width = screen_width
        self.center_x = screen_width // 2
        self.position = self.center_x
        self.win_threshold = win_threshold
        self.pull_step = 15

    def pull_left(self):
        """Move rope marker left for Player 1."""
        self.position -= self.pull_step

    def pull_right(self):
        """Move rope marker right for Player 2."""
```

```
        self.position += self.pull_step

    def reset(self):
        """Reset rope position back to exact center."""
        self.position = self.center_x

    def clamp(self):
        """Keep the rope position within visible screen bounds."""
        self.position = max(100, min(self.screen_width - 100, self.position))

    def check_winner(self):
        """Evaluate if rope crossed winning boundaries."""
        if self.position <= (self.center_x - self.win_threshold):
            return "Player 1 (Left)"
        elif self.position >= (self.center_x + self.win_threshold):
            return "Player 2 (Right)"
        return None
```

User / *Feature additions*

Now add the following features to `game/game_engine.py`:

1. A 30-second countdown timer.
2. Tie-breaker evaluation if time runs out (winner is whichever player pulled the knot closer to their side).
3. Visible threshold boundary lines (Red and Blue).
4. On-screen control text/HUD.
5. Key event 'R' to restart the match without exiting the application.

Gemini / *Countdown timer, tie-breaker, UI, and restart handler*

Here is the complete implementation of `game/game_engine.py` incorporating the timer, tie-breaker, UI elements, and restart handler:

game/game_engine.py

```
import pygame
import sys
from game.player import Player
from game.rope import Rope

class GameEngine:
    def __init__(self):
        pygame.init()
        self.width, self.height = 900, 500
        self.screen = pygame.display.set_mode((self.width, self.height))
        pygame.display.set_caption("Tug of War - Vibe Coded Edition")

        self.clock = pygame.time.Clock()
        self.fps = 60
        self.game_duration = 30  # Match timer in seconds

        # Colors
        self.WHITE = (255, 255, 255)
        self.BLACK = (20, 20, 20)
        self.RED = (220, 53, 69)
        self.BLUE = (13, 110, 253)
        self.GREEN = (40, 167, 69)
        self.GRAY = (200, 200, 200)

        # Entities
        self.p1 = Player("Player 1", pygame.K_a, self.RED, "left")
        self.p2 = Player("Player 2", pygame.K_l, self.BLUE, "right")
        self.rope = Rope(self.width)

        self.start_ticks = pygame.time.get_ticks()
        self.winner = None

        self.font = pygame.font.SysFont("Arial", 26, bold=True)
```

```
        self.large_font = pygame.font.SysFont("Arial", 42, bold=True)

    def reset(self):
        self.rope.reset()
        self.start_ticks = pygame.time.get_ticks()
        self.winner = None

    def run(self):
        running = True
        while running:
            self.clock.tick(self.fps)

            # Countdown Timer
            if self.winner is None:
                seconds_passed = (pygame.time.get_ticks() - self.start_ticks) // 1000
                time_left = max(0, self.game_duration - seconds_passed)

                if time_left == 0:
                    if self.rope.position < self.width // 2:
                        self.winner = "Player 1 (Left)"
                    elif self.rope.position > self.width // 2:
                        self.winner = "Player 2 (Right)"
                    else:
                        self.winner = "Tie Game!"
            else:
                time_left = 0

            # Inputs
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False

                if event.type == pygame.KEYDOWN:
                    if self.winner is None:
                        if self.p1.is_pulling(event):
                            self.rope.pull_left()
                        elif self.p2.is_pulling(event):
                            self.rope.pull_right()

                        self.winner = self.rope.check_winner()

                    if event.key == pygame.K_r:
                        self.reset()

            self.rope.clamp()
            self.draw(time_left)

        pygame.quit()
        sys.exit()

    def draw(self, time_left: int):
        self.screen.fill(self.WHITE)

        # Win Boundaries & Center Line
        pygame.draw.line(self.screen, self.GRAY, (self.width // 2, 80), (self.width // 2, 380), 3)
        pygame.draw.line(self.screen, self.RED, (self.width // 2 - self.rope.win_threshold, 160),
                         (self.width // 2 - self.rope.win_threshold, 340), 4)
        pygame.draw.line(self.screen, self.BLUE, (self.width // 2 + self.rope.win_threshold, 160),
                         (self.width // 2 + self.rope.win_threshold, 340), 4)

        # Rope & Knot
        pygame.draw.line(self.screen, self.BLACK, (80, 250), (self.width - 80, 250), 8)
        pygame.draw.circle(self.screen, self.GREEN, (self.rope.position, 250), 16)

        # HUD Labels
        p1_txt = self.font.render("Player 1: Press 'A'", True, self.RED)
        p2_txt = self.font.render("Player 2: Press 'L'", True, self.BLUE)
        timer_txt = self.font.render(f"Time Remaining: {time_left}s", True, self.BLACK)

        self.screen.blit(p1_txt, (40, 30))
        self.screen.blit(p2_txt, (self.width - 240, 30))
        self.screen.blit(timer_txt, (self.width // 2 - 110, 30))

        # Winner Announcement
        if self.winner:
            res_txt = self.large_font.render(f"{self.winner} Wins!", True, self.GREEN if "Tie" not in self.winner else
self.BLACK)
            rst_txt = self.font.render("Press 'R' to Restart", True, self.BLACK)
            self.screen.blit(res_txt, (self.width // 2 - res_txt.get_width() // 2, 370))
```

```
            self.screen.blit(rst_txt, (self.width // 2 - rst_txt.get_width() // 2, 430))

        pygame.display.flip()
```

User / *Entry-point request*

What should `main.py` look like?

Gemini / *main.py implementation*

Here is `main.py`, which initializes and runs `GameEngine`:

main.py

```
from game.game_engine import GameEngine

if __name__ == "__main__":
    game = GameEngine()
    game.run()
```

User / *ImportError troubleshooting*

When running `main.py`, I get this error: `ImportError: cannot import name 'Player' from 'game.player'` How do I fix this?

Gemini / *Class naming and import fix*

This error occurs when `game/player.py` defines the class name in lowercase (`player`) instead of capitalized (`Player`).

Ensure `game/player.py` contains:

game/player.py

```
import pygame

class Player:
    def __init__(self, name: str, key: int, color: tuple, side: str):
        self.name = name
        self.key = key
        self.color = color
        self.side = side

    def is_pulling(self, event: pygame.event.Event) -> bool:
        return event.type == pygame.KEYDOWN and event.key == self.key
```

Clear the compiled cache files and launch `main.py`:

Windows Command Prompt

```
rmdir /s /q game\__pycache__
python main.py
```

User / *Final status*

Done, game is working and running smoothly!